

BIO-G

Documento Maestro de Doctrina, Arquitectura y Próximos Pasos · v2

Manual largo de referencia para continuidad del proyecto. Resume la doctrina funcional del sistema, explica cómo deben conectarse hardware, semilla, calendario, entorno y múltiples BioG, y deja un mapa claro de qué conviene tocar y en qué orden.

Propósito Que el equipo pueda retomar el proyecto en cualquier chat y saber qué hace cada capa, qué verdades mandan el sistema y cuál es el siguiente orden lógico de trabajo.	Enfoque Semilla/cultivo + calendario primero. Sensores interpretados según cultivo/etapa. Cada BioG con lógica independiente.
--	---

Doctrina maestra

- La semilla/cultivo y el calendario mandan el sistema. Antes que cualquier lectura de sensores, BIO-G debe saber ****qué semilla/cultivo se colocó, cuándo se sembró o cuándo se sembrará y qué perfil fenológico aplica****.
- Cada dispositivo BioG es un sistema independiente. Si se agrega otro BioG con otra semilla, otro cultivo, otra variedad o incluso otro estado de siembra, ese dispositivo debe abrir su propio wizard y cargar ****su propia lógica****, no heredar la del anterior.
- El hardware solo cobra sentido cuando se interpreta contra el cultivo correcto. La misma humedad de suelo puede ser óptima para un cultivo y mala para otro; por eso ****los sensores nunca deben evaluarse con lógica genérica si ya se conoce el cultivo****.
- Environment alimenta contexto y decisiones, pero no debe contaminar el histórico agronómico del suelo. El clima debe subir al Dashboard y a Notifications, pero el History debe quedarse más pegado al cultivo, la etapa y el hardware.
- History debe guardar tanto eventos críticos como informativos, pero en lenguaje amigable para el agricultor. Se debe evitar jerga técnica cruda como 'V6' si el agricultor no la entiende; la app debe traducirlo a algo como 'Vegetativa avanzada'.
- Crop Care no debe ser texto estático. Debe resumir cómo ha ido el cultivo ****desde la siembra hasta la cosecha en grano****, usando el comportamiento real del Dashboard/AgroScore a lo largo del tiempo.

Regla de oro: BIO-G no debe evaluar sensores con lógica genérica si ya conoce el cultivo. La interpretación correcta siempre es: dispositivo → semilla/cultivo → calendario/etapa → lectura de sensores → score/eventos/recomendaciones.

1. Cómo debe pensarse BIO-G

BIO-G converge cuatro fuentes maestras: hardware, semilla/calendario, entorno y cuenta/contexto. La app no es un panel genérico de sensores; es un sistema de interpretación agronómica por cultivo.

La lectura de sensores solo cobra sentido cuando se sabe qué está sembrado, cuándo fue sembrado o será sembrado, y qué reglas agronómicas aplican a ese cultivo y etapa.

- **Hardware:** Humedad, temperatura, pH, EC, resistencia, NPK y demás telemetría real.
- **Semilla / Wizard / Calendario:** Perfil fenológico, fecha de siembra, variedad, calendario esperado, estimado de cosecha, crecimiento y ventanas por etapa.
- **Environment:** Clima, lluvia, forecast, temperatura ambiental y contexto externo.
- **Cuenta / Contexto:** Ubicación activa, múltiples BioG, cambio de dispositivo, relanzamiento del wizard y administración del contexto.

2. Cómo se traducen esas fuentes en salidas

Las salidas principales de la app deben estar muy bien separadas. Cada una cumple una función distinta y no conviene mezclar sus responsabilidades.

- **Dashboard:** Centro operativo en tiempo real. Debe mezclar hardware + semilla/calendario + alertas + contexto ambiental relevante.
- **History:** Memoria del cultivo. Debe mostrar telemetría y eventos importantes/informativos del cultivo, sin mezclar demasiado el clima general.
- **Crop Care / Seeds:** Resumen consolidado del estado del cultivo, no solo del momento actual. Debe usar calendario + score reciente + estabilidad + eventos.
- **Notifications:** Capa de entrega. Solo debe avisar lo que la lógica central ya decidió que merece avisarse.

3. Mapa conceptual del sistema

Esta es la lectura conceptual más útil para el proyecto ahora mismo:

[illegible]

4. Qué ya está fuerte en el código

- **`AgroScoreEngine`**: ya interpreta la telemetría según targets por etapa y produce score + suggested alerts.
- **`AlertsEngine`**: ya transforma keys sugeridas en alertas reales con cooldown anti-spam.
- **`MaizeEngine`**: ya resuelve etapa, progreso, crecimiento estimado y etiquetas amigables de la semilla.
- **`maize\_universal\_profile.dart`**: ya contiene la biología operativa del maíz por etapa.

- **‘BioGStore’**: ya actúa como hub global entre repository y UI.
- **‘EnvironmentService’**: ya permite que el módulo clima sea relativamente independiente.

5. Qué todavía no está totalmente cerrado

- History ya tiene UI de eventos, pero todavía falta amarrar la lógica y persistencia del timeline de eventos del cultivo.
- Seeds/Crop Care todavía necesita consolidarse como un resumen del desempeño histórico del cultivo, no solo del calendario.
- Dashboard hoy calcula mucho; a futuro conviene que consuma más resultados consolidados y menos lógica distribuida por pantalla.
- Multi-BioG todavía necesita una política más formal: cada dispositivo debe abrir wizard propio y cargar lógica propia de cultivo.
- Environment aún no se refleja del todo como feed contextual del Dashboard y notificaciones.

6. Diferencia entre alerta, evento y recomendación

- **Alerta**: algo urgente o fuera de rango. Ejemplo: humedad crítica.
- **Evento**: algo relevante del ciclo o del contexto. Ejemplo: entró a floración, viene lluvia, recuperación sostenida.
- **Recomendación**: acción sugerida. Ejemplo: regar, revisar compactación, monitorear nitrógeno.

7. Decisiones de proyecto ya definidas

- History debe guardar ****eventos críticos e informativos****.
- Los eventos de History deben salir en ****lenguaje amigable para el agricultor****.
- Environment detectando lluvia próxima debe verse en ****Dashboard y Notifications****, no en History.
- Cada BioG debe tener ****su propio contexto de semilla/cultivo****.
- Si agregas otro BioG, el sistema debe abrir ****wizard obligatorio**** para registrar qué va a sembrarse ahí.
- Debe existir un perfil genérico/skip, pero no debe ser la ruta preferida.
- El sistema debe poder representar estados como ‘aún no sembrado’, por ejemplo si la siembra será en tres días.

8. Crop Care: cómo debe pensarse

Crop Care debe resumir el comportamiento del cultivo desde la siembra hasta la cosecha, no solo describir la etapa actual.

La forma recomendada es usar una combinación de calendario, score reciente, estabilidad y eventos acumulados.

```
Crop Care =
estado biológico esperado por calendario
+ desempeño reciente del AgroScore
+ estabilidad de métricas en el tiempo
+ severidad y frecuencia de eventos
+ peso de la etapa actual
```

9. Orden recomendado de implementación

- **Prioridad 1:** History events + modelo común de evento.
- **Prioridad 2:** Crop Care real en Seeds, alimentado por score/historial/eventos.
- **Prioridad 3:** Feed contextual del Dashboard con eventos y entorno.
- **Prioridad 4:** Reglas claras para Notifications.
- **Prioridad 5:** Formalización completa de multi-BioG y wizard por dispositivo.

10. Lectura pantalla por pantalla

Dashboard

Es el centro operativo en tiempo real. Aquí vive la lectura más inmediata del BioG activo.

Debe mezclar hardware + semilla/calendario + alertas + contexto ambiental útil.

No debería quedarse como simple pantalla 'que dibuja'; debe ser consumidora de una verdad agronómica coherente.

History

Es la memoria del cultivo. Debe mostrar series históricas por métrica y, debajo, un timeline de eventos del cultivo.

Los eventos deben ser críticos e informativos, pero con lenguaje amigable para el agricultor.

La lluvia próxima no necesita vivir aquí; History debe quedarse más limpio y más pegado al cultivo.

Seeds / Crop Care

Debe ser el resumen consolidado del cultivo por dispositivo.

Aquí convergen semilla, fecha de siembra, perfil, etapa, crecimiento esperado, estimado de cosecha y cuidado del cultivo.

El cuidado del cultivo no debe salir solo del calendario; debe resumir cómo se ha comportado el score del cultivo a lo largo del tiempo.

NPK

Es la vista profunda del estado nutrimental. Recalcula targets por etapa y contrasta contra el historial reciente.

Cuando se toquen fórmulas del AgroScore o targets universales, hay que revisar Dashboard y NPK juntos para evitar incoherencias.

Environment

Debe permanecer relativamente desacoplado del núcleo de hardware.

Su rol es alimentar contexto útil hacia Dashboard y Notifications, no convertirse en dueño del historial del cultivo.

Account

Es una capa de contexto y orquestación. Desde aquí deben poder cambiar ubicación, dispositivos y, más adelante, relanzar wizard por BioG.

Cada BioG agregado debe terminar con su propio contexto de cultivo, idealmente guiado por wizard obligatorio.

11. Inventario archivo por archivo

Las siguientes fichas sirven como mapa rápido para saber dónde entrar cuando algo falle o cuando se quiera mover una parte de la app.

core/agro/agro_score_engine.dart	283 líneas
Rol Motor central que interpreta telemetría según etapa y perfil universal.	Qué mueve Normalización, score del ring, bandas por métrica, NPK indexado y suggested alerts.
Cuándo tocarlo Cambiar fórmula del ring, rangos, pesos o interpretación de sensores.	Cuidado Una modificación aquí cambia dashboard, NPK, alertas y lectura agronómica global.
Clases AgroScoreEngine, _Eval	Depende de core/agro/agro_types.dart, core/agro/alerts_engine.dart, crops/maize/maize_universal_profile.dart, models/biog_telemetry.dart
Lo usan —	
core/agro/agro_types.dart	114 líneas
Rol Tipos base del motor agronómico: métricas, bandas, rangos, evaluación, calibración y estado de alertas.	Qué mueve El lenguaje común entre score, alerts, dashboard y NPK.
Cuándo tocarlo Agregar nuevas métricas, nuevos estados o cambiar estructuras de resultado.	Cuidado Cambios aquí se sienten en todo el core agro.
Clases AgroRange, AgroMetricEval, AgroEvalResult, Calibration, AlertsState, AlertsBuildResult	Depende de models/biog_telemetry.dart
Lo usan core/agro/agro_score_engine.dart	
core/agro/alerts_engine.dart	177 líneas
Rol Traduce keys sugeridas a BioGAlert reales y aplica cooldown anti-spam.	Qué mueve Severidad, tipo, mensaje, deduplicación y ritmo de alertas.
Cuándo tocarlo Cambiar lenguaje de alertas, prioridad o política anti-spam.	Cuidado Buen lugar para centralizar alertas; no repartir esta lógica en la UI.
Clases AlertsEngine, _AlertTemplate	Depende de core/agro/agro_types.dart, models/biog_telemetry.dart, widgets/seeds/maize_models.dart
Lo usan core/agro/agro_score_engine.dart	

crops/maize/maize_universal_profile.dart	615 líneas
Rol Perfil universal de maíz por etapa. Define targets y pesos del score.	Qué mueve Biología operativa del cultivo: humedad, suelo, NPK, sensibilidad y ponderación por etapa.
Cuándo tocarlo Refinar agronomía, calendario y pesos por etapa.	Cuidado Archivo de dominio muy influyente; tratarlo como “verdad del cultivo”.
Clases MaizeUniversalProfile, StageTargets, StageWeights	Depende de core/agro/agro_types.dart, widgets/seeds/maize_models.dart
Lo usan core/agro/agro_score_engine.dart	
main.dart	56 líneas
Rol Punto de arranque. Inicializa Flutter, tema, repositorio demo y BioGStore; luego monta la app con BioGScope.	Qué mueve Bootstrapping global, tema principal, inyección de dependencias, shell inicial.
Cuándo tocarlo Cambio de arranque, cambio de proveedor global, cambio de ruta/pantalla inicial.	Cuidado Si falla aquí, casi toda la app deja de levantar.
Clases BioGApp	Depende de screens/app_shell.dart, services/biog/biog_store.dart, services/biog/fake_biog_repository.dart, theme/app_theme.dart
Lo usan —	
models/biog_telemetry.dart	161 líneas
Rol Modelos del dominio principal: dispositivo, telemetría y alerta.	Qué mueve El vocabulario común entre repo, store, engines y UI.
Cuándo tocarlo Cambios de payload de hardware o estructura de alertas.	Cuidado Cambios aquí impactan muchas capas.
Clases BioGDevice, BioGTelemetry, BioGAlert	Depende de —
Lo usan core/agro/agro_score_engine.dart, core/agro/agro_types.dart, core/agro/alerts_engine.dart	

models/environment_models.dart	259 líneas
Rol Modelos del módulo de entorno y forecast.	Qué mueve Condición actual, diarios, insights y payload ambiental.
Cuándo tocarlo Cambios del proveedor meteorológico o nuevas variables.	Cuidado Mantenerlo desacoplado del núcleo de hardware.
Clases EnvironmentIconMapper, EnvironmentLocation, EnvironmentNow, EnvironmentDaily, EnvironmentInsight, EnvironmentPayload	Depende de —
Lo usan —	
models/seed_install.dart	25 líneas
Rol Modelo mínimo de semilla instalada en un BioG.	Qué mueve Fecha de siembra, alias y profileId opcional.
Cuándo tocarlo Formalizar wizard, instalación de cultivo o multi-dispositivo.	Cuidado Crecerá cuando el wizard sea más formal.
Clases SeedInstall	Depende de —
Lo usan —	
screens/account_screen.dart	1246 líneas
Rol Pantalla de cuenta y configuración del contexto general.	Qué mueve Perfil, ubicación, gestión de BioG y acceso a subflujos.
Cuándo tocarlo Cambiar experiencia de cuenta, añadir manejo real de múltiples BioG.	Cuidado Archivo grande; conviene dividir más si sigue creciendo.
Clases AccountScreen, _AccountScreenState, _BioGSoftBackground, _GlowBlob, _GlassCard, _AvatarCircle	Depende de widgets/account/add_biog_screen.dart, widgets/account/edit_profile_screen.dart, widgets/account/status_biog_screen.dart, widgets/bottom_nav.dart
Lo usan —	

screens/app_shell.dart		94 líneas
Rol Shell principal con IndexedStack y bottom nav. Mantiene vivas las pantallas base.	Qué mueve Orden de tabs, navegación primaria, tab forzado en desarrollo.	
Cuándo tocarlo Agregar/quitar pantalla principal o cambiar flujo entre tabs.	Cuidado IndexedStack conserva estado y listeners; lo que viva aquí sigue vivo en background.	
Clases AppShell, _AppShellState	Depende de screens/account_screen.dart, screens/dashboard_screen.dart, screens/environment/environment_screen.dart, screens/history_screen.dart	
Lo usan main.dart		
screens/dashboard_screen.dart		707 líneas
Rol Centro operativo en tiempo real.	Qué mueve Lee live telemetry y seed activa; resuelve etapa, corre score y pinta ring, métricas e insights.	
Cuándo tocarlo Cambios de layout, cards, feed contextual o navegación a NPK.	Cuidado Hoy mezcla UI con parte de la lógica. Es una zona sensible.	
Clases DashboardScreen, _Header	Depende de core/agro/agro_score_engine.dart, core/agro/agro_types.dart, crops/maize/maize_universal_profile.dart, models/biog_telemetry.dart	
Lo usan screens/app_shell.dart		
screens/environment/environment_forecast_24h_screen.dart		808 líneas
Rol Pantalla secundaria del módulo Environment.	Qué mueve Amplía o detalla el flujo de clima y forecast.	
Cuándo tocarlo Cuando el módulo donde vive cambie de UX, datos o navegación.	Cuidado Revisar dependencias y no moverlo aislado del resto de su módulo.	
Clases EnvironmentForecast24hScreenDark, _EnvironmentForecast24hScreenDarkState, _HourRow, _HourRowCard, _SkeletonHourRow, _StaggerIn	Depende de models/environment_models.dart, services/environment_service.dart, widgets/environment/environment_location_card.dart, widgets/shared/bio_g_glass_card.dart	
Lo usan —		

screens/environment/environment_forecast_screen.dart	804 líneas
Rol Pantalla secundaria del módulo Environment.	Qué mueve Amplía o detalla el flujo de clima y forecast.
Cuándo tocarlo Cuando el módulo donde vive cambie de UX, datos o navegación.	Cuidado Revisar dependencias y no moverlo aislado del resto de su módulo.
Clases EnvironmentForecastScreen, _EnvironmentForecastScreenState, _DayRowCard, _SkeletonDayRow, _StaggerIn, _ForecastIcon	Depende de models/environment_models.dart, screens/environment/environment_forecast_24h_screen.dart, services/environment_service.dart, widgets/environment/environment_location_card.dart
Lo usan —	
screens/environment/environment_screen.dart	511 líneas
Rol Módulo principal de entorno.	Qué mueve Forecast, métricas ambientales, ubicación y resumen actual.
Cuándo tocarlo Cambiar UX de clima o integrar mejor contexto al Dashboard.	Cuidado Debe alimentar Dashboard/Notifications, no contaminar History.
Clases EnvironmentScreen, _EnvironmentScreenState, _EnvironmentSoftBackground, _GlowBlob, _LoadingBody, _ErrorBody	Depende de models/environment_models.dart, screens/environment/environment_forecast_screen.dart, services/environment_service.dart, widgets/bottom_nav.dart
Lo usan screens/app_shell.dart	
screens/history_screen.dart	623 líneas
Rol Memoria del cultivo basada en telemetría e historial.	Qué mueve Series por métrica, ventanas 24h/7d/30d/todo y lista de eventos.
Cuándo tocarlo Conectar eventos reales, mejorar timeline y cambiar UX de View All.	Cuidado Hoy tiene hueco importante: eventos todavía no totalmente consolidados.
Clases HistoryScreen, _HistoryScreenState, HistorySeries, _HistoryTopBar, _RangeSubtitle, _IconChip	Depende de models/biog_telemetry.dart, services/biog/biog_store.dart, widgets/bottom_nav.dart, widgets/history/history_chart_card.dart
Lo usan screens/app_shell.dart	

screens/npk/npk_screen.dart	1172 líneas
Rol Vista profunda de N, P y K con gauges, targets y tendencia.	Qué mueve Recalcula contexto de etapa y contrasta historial NPK con targets.
Cuándo tocarlo Ajustar gauges, insight o escala operativa NPK.	Cuidado Revisar duplicación de lógica con Dashboard cuando se toquen fórmulas.
Clases NpkScreen, _NpkStats, _NpkTabsCard, _NpkContentCardShell, _NpkTabContent, _StageInsightPill	Depende de core/agro/agro_types.dart, crops/maize/maize_universal_profile.dart, models/biog_telemetry.dart, services/biog/biog_store.dart
Lo usan screens/dashboard_screen.dart	
screens/seeds_screen.dart	1008 líneas
Rol Pantalla de cultivo/semillas y puente hacia crop care.	Qué mueve Etapa, calendario, estimados, hero visual y cuidado del cultivo.
Cuándo tocarlo Formalizar crop care, wizard o semántica del cultivo por dispositivo.	Cuidado Debe consumir tanto semilla/calendario como comportamiento real reciente.
Clases SeedsScreen, _Estimation, SeedsScreenArgs, _CompactFieldCard, _StageContainerCard, _CareCardInner	Depende de services/biog/biog_store.dart, widgets/bottom_nav.dart, widgets/seeds/maize_engine.dart, widgets/seeds/maize_models.dart
Lo usan screens/app_shell.dart	
services/biog/biog_repository.dart	35 líneas
Rol Contrato de datos de dispositivos, telemetría, historial y alertas.	Qué mueve La frontera entre la capa de datos y el resto del sistema.
Cuándo tocarlo Migración a hardware real o cambio de fuente de datos.	Cuidado Todo repositorio concreto debe respetar este contrato.
Clases BioGRepository	Depende de models/biog_telemetry.dart
Lo usan —	

services/biog/biog_store.dart		151 líneas
Rol Estado global observable. Escucha el repositorio y guarda snapshots rápidos para UI.	Qué mueve Dispositivo activo, telemetría viva, alertas recientes, seed activa, alertsState y lastAgroEval.	
Cuándo tocarlo Nueva pieza de estado global o helper transversal entre pantallas.	Cuidado Debe orquestar, no absorber toda la lógica de negocio.	
Clases BioGStore, BioGScope	Depende de core/agro/agro_types.dart, models/biog_telemetry.dart, models/seed_install.dart, services/biog/biog_repository.dart	
Lo usan main.dart, screens/dashboard_screen.dart, screens/history_screen.dart, screens/npk/npk_screen.dart		
services/biog/fake_biog_repository.dart		464 líneas
Rol Simula hardware, genera telemetría viva, historial y alertas para desarrollo.	Qué mueve Tick de simulación, creación de demo devices, history fake, alertas fake y contexto agronómico demo.	
Cuándo tocarlo Ajustar demo, probar escenarios extremos o conectar mejor múltiples BioG.	Cuidado Hoy carga muchas responsabilidades; útil para demo, delicado para producción.	
Clases FakeBioGRepository	Depende de core/agro/agro_score_engine.dart, core/agro/agro_types.dart, crops/maize/maize_universal_profile.dart, models/biog_telemetry.dart	
Lo usan main.dart		
services/environment_service.dart		329 líneas
Rol Servicio HTTP del módulo de entorno.	Qué mueve Llamadas al proveedor, parsing y generación de insights ambientales.	
Cuándo tocarlo Cambiar endpoint, parsing o reglas de insight.	Cuidado Mantener lejos de widgets.	
Clases EnvironmentService	Depende de models/environment_models.dart	
Lo usan screens/environment/environment_forecast_24h_screen.dart, screens/environment/environment_forecast_screen.dart, screens/environment/environment_screen.dart		

theme/app_theme.dart		21 líneas
Rol Archivo de tema o estilos globales.	Qué mueve Define colores, tipografía o tokens visuales.	
Cuándo tocarlo Cuando el módulo donde vive cambie de UX, datos o navegación.	Cuidado Revisar dependencias y no moverlo aislado del resto de su módulo.	
Clases AppTheme	Depende de —	
Lo usan main.dart		
theme/bio_g_theme.dart		92 líneas
Rol Archivo de tema o estilos globales.	Qué mueve Define colores, tipografía o tokens visuales.	
Cuándo tocarlo Cuando el módulo donde vive cambie de UX, datos o navegación.	Cuidado Revisar dependencias y no moverlo aislado del resto de su módulo.	
Clases BioGTheme	Depende de —	
Lo usan —		
widgets/account/add_biog_screen.dart		499 líneas
Rol Subpantalla o widget del flujo de Account.	Qué mueve Compone una parte del flujo de perfil, ubicación, QR o estado de BioG.	
Cuándo tocarlo Cuando el módulo donde vive cambie de UX, datos o navegación.	Cuidado Revisar dependencias y no moverlo aislado del resto de su módulo.	
Clases AddBioGScreen, _BioGLocalStore, _SectionCard, _ActionTile, _ActionTileState, _GlassCard	Depende de widgets/account/bluetooth_scan_screen.dart, widgets/account/qr_scan_screen.dart	
Lo usan screens/account_screen.dart		

widgets/account/bluetooth_scan_screen.dart	303 líneas
Rol Subpantalla o widget del flujo de Account.	Qué mueve Compone una parte del flujo de perfil, ubicación, QR o estado de BioG.
Cuándo tocarlo Cuando el módulo donde vive cambie de UX, datos o navegación.	Cuidado Revisar dependencias y no moverlo aislado del resto de su módulo.
Clases BluetoothScanScreen, _BluetoothScanScreenState, _DeviceTile, _DividerLine, _GlassCard, _SoftBackground	Depende de —
Lo usan widgets/account/add_biog_screen.dart	
widgets/account/edit_profile_screen.dart	1658 líneas
Rol Subpantalla o widget del flujo de Account.	Qué mueve Compone una parte del flujo de perfil, ubicación, QR o estado de BioG.
Cuándo tocarlo Cuando el módulo donde vive cambie de UX, datos o navegación.	Cuidado Revisar dependencias y no moverlo aislado del resto de su módulo.
Clases EditProfileScreen, _EditProfileScreenState, _TopBar, _SavePillButton, _SavePillButtonState, _HeaderCard	Depende de widgets/account/location_screen.dart, widgets/account/telephone_screen.dart
Lo usan screens/account_screen.dart	
widgets/account/location_screen.dart	938 líneas
Rol Subpantalla o widget del flujo de Account.	Qué mueve Compone una parte del flujo de perfil, ubicación, QR o estado de BioG.
Cuándo tocarlo Cuando el módulo donde vive cambie de UX, datos o navegación.	Cuidado Revisar dependencias y no moverlo aislado del resto de su módulo.
Clases LocationScreen, _LocationScreenState, _TopSavePill, _TopSavePillState, _AssetIcon, _SearchBar	Depende de —
Lo usan widgets/account/edit_profile_screen.dart	

widgets/account/qr_scan_screen.dart	285 líneas
Rol Subpantalla o widget del flujo de Account.	Qué mueve Compone una parte del flujo de perfil, ubicación, QR o estado de BioG.
Cuándo tocarlo Cuando el módulo donde vive cambie de UX, datos o navegación.	Cuidado Revisar dependencias y no moverlo aislado del resto de su módulo.
Clases QrScanScreen, _PrimaryButton, _GlassCard, _SoftBackground, _GlowBlob	Depende de —
Lo usan widgets/account/add_biog_screen.dart	
widgets/account/status_biog_screen.dart	647 líneas
Rol Subpantalla o widget del flujo de Account.	Qué mueve Compone una parte del flujo de perfil, ubicación, QR o estado de BioG.
Cuándo tocarlo Cuando el módulo donde vive cambie de UX, datos o navegación.	Cuidado Revisar dependencias y no moverlo aislado del resto de su módulo.
Clases StatusBioGScreen, _BioGLocalStore, _TopBar, _ChipPill, _StatusRowAssetPlain, _DividerLine	Depende de —
Lo usan screens/account_screen.dart	
widgets/account/telephone_screen.dart	1443 líneas
Rol Subpantalla o widget del flujo de Account.	Qué mueve Compone una parte del flujo de perfil, ubicación, QR o estado de BioG.
Cuándo tocarlo Cuando el módulo donde vive cambie de UX, datos o navegación.	Cuidado Revisar dependencias y no moverlo aislado del resto de su módulo.
Clases TelephoneScreen, _CountryMeta, _TelephoneScreenState, _TopSavePill, _TopSavePillState, _CountryPill	Depende de —
Lo usan widgets/account/edit_profile_screen.dart	

widgets/bottom_nav.dart	373 líneas
Rol Bottom nav custom de BIO-G con CTA central.	Qué mueve Navegación primaria y gramática visual base del shell.
Cuándo tocarlo Cambiar tabs, iconografía o comportamiento del CTA.	Cuidado Se siente en toda la app.
Clases BioGBottomNav, _NavItem, _CenterCTATapTarget, _CenterCTA, _BioGWaveNotchClipper	Depende de —
Lo usan screens/account_screen.dart, screens/dashboard_screen.dart, screens/environment/environment_screen.dart, screens/history_screen.dart	
widgets/environment/environment_forecast_preview_card.dart	228 líneas
Rol Widget del módulo Environment.	Qué mueve Presenta tarjetas y bloques de clima/forecast.
Cuándo tocarlo Cuando el módulo donde vive cambie de UX, datos o navegación.	Cuidado Revisar dependencias y no moverlo aislado del resto de su módulo.
Clases EnvironmentForecastPreviewCard, _DayCol	Depende de models/environment_models.dart, widgets/shared/bio_g_glass_card.dart
Lo usan screens/environment/environment_screen.dart	
widgets/environment/environment_insight_card.dart	73 líneas
Rol Widget del módulo Environment.	Qué mueve Presenta tarjetas y bloques de clima/forecast.
Cuándo tocarlo Cuando el módulo donde vive cambie de UX, datos o navegación.	Cuidado Revisar dependencias y no moverlo aislado del resto de su módulo.
Clases EnvironmentInsightCard	Depende de models/environment_models.dart
Lo usan screens/environment/environment_screen.dart	

widgets/environment/environment_location_card.dart	172 líneas
Rol Widget del módulo Environment.	Qué mueve Presenta tarjetas y bloques de clima/forecast.
Cuándo tocarlo Cuando el módulo donde vive cambie de UX, datos o navegación.	Cuidado Revisar dependencias y no moverlo aislado del resto de su módulo.
Clases EnvironmentLocationCard, _RowLine, _AssetIcon	Depende de widgets/shared/bio_g_glass_card.dart
Lo usan screens/environment/environment_forecast_24h_screen.dart, screens/environment/environment_forecast_screen.dart, screens/environment/environment_screen.dart	
widgets/environment/environment_metric_tile.dart	200 líneas
Rol Widget del módulo Environment.	Qué mueve Presenta tarjetas y bloques de clima/forecast.
Cuándo tocarlo Cuando el módulo donde vive cambie de UX, datos o navegación.	Cuidado Revisar dependencias y no moverlo aislado del resto de su módulo.
Clases EnvironmentMetricTile, _IconBox, _ChipPill	Depende de widgets/shared/bio_g_glass_card.dart
Lo usan screens/environment/environment_screen.dart	
widgets/environment/environment_now_summary_card.dart	153 líneas
Rol Widget del módulo Environment.	Qué mueve Presenta tarjetas y bloques de clima/forecast.
Cuándo tocarlo Cuando el módulo donde vive cambie de UX, datos o navegación.	Cuidado Revisar dependencias y no moverlo aislado del resto de su módulo.
Clases EnvironmentNowSummaryCard, _BigIcon	Depende de models/environment_models.dart, widgets/shared/bio_g_glass_card.dart
Lo usan screens/environment/environment_forecast_screen.dart, screens/environment/environment_screen.dart	

widgets/history/history_chart_card.dart	924 líneas
Rol Widget del módulo History.	Qué mueve Presenta una parte del histórico o de los eventos.
Cuándo tocarlo Cuando el módulo donde vive cambie de UX, datos o navegación.	Cuidado Revisar dependencias y no moverlo aislado del resto de su módulo.
Clases HistoryChartCard, _BandStatus, _BubbleEntrance, _LiquidPillBubble, _ChevronPointer, _ChevronPainter	Depende de widgets/shared/bio_g_glass_card.dart
Lo usan screens/history_screen.dart	
widgets/history/history_event_tile.dart	168 líneas
Rol Widget del módulo History.	Qué mueve Presenta una parte del histórico o de los eventos.
Cuándo tocarlo Cuando el módulo donde vive cambie de UX, datos o navegación.	Cuidado Revisar dependencias y no moverlo aislado del resto de su módulo.
Clases HistoryEventTile, _EventIconBase, _MiniDots	Depende de —
Lo usan —	
widgets/history/history_events_list.dart	116 líneas
Rol Widget del módulo History.	Qué mueve Presenta una parte del histórico o de los eventos.
Cuándo tocarlo Cuando el módulo donde vive cambie de UX, datos o navegación.	Cuidado Revisar dependencias y no moverlo aislado del resto de su módulo.
Clases HistoryEventsList, _ExportChip	Depende de widgets/history/history_event_tile.dart, widgets/shared/bio_g_glass_card.dart
Lo usan screens/history_screen.dart	

widgets/history/history_metric_tabs.dart	360 líneas
Rol Widget del módulo History.	Qué mueve Presenta una parte del histórico o de los eventos.
Cuándo tocarlo Cuando el módulo donde vive cambie de UX, datos o navegación.	Cuidado Revisar dependencias y no moverlo aislado del resto de su módulo.
Clases HistoryMetricTabs, _HistoryMetricTabsState, _PillBar, _PillBarState, _TapTab, _TapTabState	Depende de —
Lo usan screens/history_screen.dart	
widgets/history/history_npk_chart_card.dart	908 líneas
Rol Widget del módulo History.	Qué mueve Presenta una parte del histórico o de los eventos.
Cuándo tocarlo Cuando el módulo donde vive cambie de UX, datos o navegación.	Cuidado Revisar dependencias y no moverlo aislado del resto de su módulo.
Clases HistoryNpkChartCard, _NpkState, _StatusInlineDot, _LastReadingRow, _NpkWideChips, _ScaledAssetIcon	Depende de widgets/shared/bio_g_glass_card.dart
Lo usan screens/history_screen.dart	
widgets/history/history_range_selector.dart	363 líneas
Rol Widget del módulo History.	Qué mueve Presenta una parte del histórico o de los eventos.
Cuándo tocarlo Cuando el módulo donde vive cambie de UX, datos o navegación.	Cuidado Revisar dependencias y no moverlo aislado del resto de su módulo.
Clases HistoryRangeSelector, _HistoryRangeSelectorState, _RangePillBar, _RangePillBarState, _TapPill, _TapPillState	Depende de —
Lo usan screens/history_screen.dart	

widgets/insight_card.dart		245 líneas
Rol Archivo del proyecto BIO-G.	Qué mueve Participa dentro de la arquitectura actual.	
Cuándo tocarlo Cuando el módulo donde vive cambie de UX, datos o navegación.	Cuidado Revisar dependencias y no moverlo aislado del resto de su módulo.	
Clases InsightCard, _InsightIconBase, _TagChip	Depende de —	
Lo usan screens/dashboard_screen.dart		
widgets/metric_card.dart		312 líneas
Rol Archivo del proyecto BIO-G.	Qué mueve Participa dentro de la arquitectura actual.	
Cuándo tocarlo Cuando el módulo donde vive cambie de UX, datos o navegación.	Cuidado Revisar dependencias y no moverlo aislado del resto de su módulo.	
Clases MetricCard, _MetricIcon, _StatusPill, _MetricDots, _MetricStyle	Depende de —	
Lo usan screens/dashboard_screen.dart		
widgets/npk/npk_gauge_card.dart		407 líneas
Rol Archivo del proyecto BIO-G.	Qué mueve Participa dentro de la arquitectura actual.	
Cuándo tocarlo Cuando el módulo donde vive cambie de UX, datos o navegación.	Cuidado Revisar dependencias y no moverlo aislado del resto de su módulo.	
Clases NpkGaugeCard, _GaugeCenter, _NpkGaugePainter	Depende de —	
Lo usan screens/npk/npk_screen.dart		

widgets/npk/npk_models.dart		2 líneas
Rol Archivo del proyecto BIO-G.		Qué mueve Participa dentro de la arquitectura actual.
Cuándo tocarlo Cuando el módulo donde vive cambie de UX, datos o navegación.		Cuidado Revisar dependencias y no moverlo aislado del resto de su módulo.
Clases —		Depende de —
Lo usan —		
widgets/npk_insight_card.dart		166 líneas
Rol Archivo del proyecto BIO-G.		Qué mueve Participa dentro de la arquitectura actual.
Cuándo tocarlo Cuando el módulo donde vive cambie de UX, datos o navegación.		Cuidado Revisar dependencias y no moverlo aislado del resto de su módulo.
Clases NpkInsightCard, _NpkInsightCardState		Depende de —
Lo usan screens/dashboard_screen.dart		
widgets/seeds/maize_engine.dart		227 líneas
Rol Resuelve etapa, progreso, ventanas, crecimiento y hero asset según fecha de siembra y perfil.		Qué mueve Puente entre semilla/calendario y toda la lectura agronómica.
Cuándo tocarlo Ajustar transiciones de etapa o reglas del calendario.		Cuidado Si la etapa está mal, todo lo demás se desalineará.
Clases MaizeEngine		Depende de widgets/seeds/maize_models.dart
Lo usan screens/dashboard_screen.dart, screens/npk/npk_screen.dart, screens/seeds_screen.dart, services/biog/fake_biog_repository.dart		

widgets/seeds/maize_models.dart		130 líneas
Rol Modelos puros de fenología y perfiles de maíz.	Qué mueve Etapas, ventanas, rangos y resultado de etapa.	
Cuándo tocarlo Agregar detalle fenológico o ampliar modelos de perfil.	Cuidado No depende de Flutter; mantenerlo limpio y reusable.	
Clases RangeInt, RangeDouble, MaizeProfile, StageBounds, SeedStageResult	Depende de —	
Lo usan core/agro/agro_score_engine.dart, core/agro/alerts_engine.dart, crops/maize/maize_universal_profile.dart, screens/dashboard_screen.dart		
widgets/seeds/maize_profiles.dart		37 líneas
Rol Catálogo de perfiles y alias de maíz.	Qué mueve Traducción de variedad/alias hacia perfil fenológico concreto.	
Cuándo tocarlo Agregar semillas, perfiles o alias comerciales.	Cuidado Si una semilla no “cae” en el perfil correcto, revisar aquí primero.	
Clases —	Depende de widgets/seeds/maize_models.dart	
Lo usan screens/dashboard_screen.dart, screens/npk/npk_screen.dart, screens/seeds_screen.dart, services/biog/fake_biog_repository.dart		
widgets/shared/bio_g_button.dart		282 líneas
Rol Widget/shared reusable.	Qué mueve Base visual reutilizable para varias pantallas.	
Cuándo tocarlo Cuando el módulo donde vive cambie de UX, datos o navegación.	Cuidado Revisar dependencias y no moverlo aislado del resto de su módulo.	
Clases BioGButton, _BioGButtonState	Depende de —	
Lo usan screens/account_screen.dart		

widgets/shared/bio_g_glass_card.dart	90 líneas
Rol Widget/shared reusable.	Qué mueve Base visual reutilizable para varias pantallas.
Cuándo tocarlo Cuando el módulo donde vive cambie de UX, datos o navegación.	Cuidado Revisar dependencias y no moverlo aislado del resto de su módulo.
Clases BioGGlassCard	Depende de —
Lo usan screens/environment/environment_forecast_24h_screen.dart, screens/environment/environment_forecast_screen.dart, screens/seeds_screen.dart, widgets/environment/environment_forecast_preview_card.dart	
widgets/soil_health_ring.dart	513 líneas
Rol Representación visual del Soil Health Ring.	Qué mueve Animación, painter, badge y estética del score.
Cuándo tocarlo Ajustar look/feel del ring.	Cuidado No calcula nada; solo representa el score.
Clases SoilHealthRing, _SoilHealthRingState, _OuterFrameRingPainter, _BaseRingPainter, _ActiveRingPainter, _RingBadge	Depende de —
Lo usan screens/dashboard_screen.dart	

12. Mapa de batalla recomendado

Con la doctrina ya definida, el siguiente orden más sano de trabajo es este:

- Formalizar un `AgronomicEvent` común para history/dashboard/notifications.
- Rediseñar History Events List para 'View All' y timeline expandible.
- Convertir Crop Care en resumen del cultivo desde siembra hasta cosecha.
- Pegar Environment al Dashboard como feed contextual, no al History.
- Definir multi-BioG como contextos independientes con wizard obligatorio.

13. Prompt maestro para continuar en otros chats

Este bloque sirve para copiar y pegar en un chat nuevo sin volver a explicar toda la arquitectura:

```
Estamos trabajando en BIO-G. El sistema debe estar centrado primero en la semilla/cultivo y su calendario, no en sensores genéricos. Cada BioG tiene su propio contexto independiente de semilla, wizard y lógica de cultivo. Environment alimenta Dashboard y Notifications, pero no History. History debe guardar eventos críticos e informativos en lenguaje amigable para agricultor. Crop Care debe resumir el comportamiento del cultivo desde siembra hasta cosecha usando calendario + score + estabilidad + eventos. Quiero continuar desde el documento maestro BIOG_Master_Doctrine_and_Roadmap_v2 y seguir con la siguiente prioridad del roadmap sin romper la arquitectura base.
```